

Programming a Robot - Solution

Chapter 5: "A more complicated situation"

The only difference between this situation and the example situation from chapter 4 is, that our robot is now set to face up instead of right in the beginning, so we just have to add an instruction that turns our robot right. After that, we can reuse the program from before.

Location	Contents	Meaning
0	26	trn 6
1	13	fwd 3
2	22	trn 2
3	11	fwd 1
4	00	hlt

Chapter 5 "An even more complicated map"

In this scenario, our robot walks five steps and then turns left. It then walks 4 steps before turning right and walks another three steps. Finally, it just turns right, walks one step forward and then stops on the marked tile.

Location	Contents	Meaning
0	26	trn 6
1	15	fwd 5
2	22	trn 2
3	14	fwd 4
4	26	trn 6
5	13	fwd 3
6	26	trn 6
7	11	fwd 1
8	00	hlt

Chapter 6 "Can you see a way to make the program only execute 3 instructions in the loop instead of four and still go around in a circle?"

If we look at the program, we can see that the following happens: In the beginning the robot turns left, walks three steps and then turns towards the tile it just came from. The program then jumps back to the beginning and the robot turns right before walking forward. While the robot needs to turn

left before it starts walking, it does not need to execute the turn to the next side of the square in two instructions.

Location	Contents	Meaning
0	22	trn 2
1	13	fwd 3
2	26	trn 6
3	31	jmp 1

Chapter 6 “Can you devise a program that will make the robot move around the square forever in an anticlockwise direction?”

Since our robot is already facing the right way, this time it can start walking first, it then needs to turn left and then our program repeats.

Location	Contents	Meaning
0	13	fwd 3
1	22	trn 2
2	30	jmp 0

Chapter 7 “Can you find the test condition that causes the strange behaviour?”

The robot first walks to the next corner before checking if it is standing on the target, so if the target is in the first corner and the robot is already standing on it, it will first test all other corners before coming back and stopping on the target.

Chapter 7 “Can you make a program that will drive the robot the right distance to the target, no matter where it is?”

In this case we need the robot to do the following: Check if it is standing on the target, walk a step and check again. It is very important to check before walking, since the target could already be the tile the robot is standing on.

Location	Contents	Meaning
0	43	jlt 3
1	11	fwd 1
2	51	jnt 1
3	00	hlt

Chapter 7 “Does your program work correctly if the robot is already on the target when the program starts running?”

If that is not the case try implementing a jump that checks whether the robot is on the target before it takes the first step.

Chapter 8 “Can you see how to do it?”

Here we combine the ideas from the previous chapters, we want the robot to alternately check for a wall in front of it and walk if there is none. Otherwise, we want it to turn left, since there are only left turns in this map until it reaches the target.

Location	Contents	Meaning
0	46	jit
1	64	jiw
2	11	fwd 1
3	30	jmp 0
4	22	trn 2
5	30	jmp 0
6	00	hlt

Chapter 8 “What does your program do when confronted with this map?”

This map also contains right turns, while our previous program can only turn left. So yes, the robot will get stuck.

Chapter 9 “Can you write a program to do this job using the machine instructions?”

This program is a bit challenging, especially since we have only space for ten numbers. However, it is still possible. We combine everything we have learned so far: We want the robot to alternately walk and check in what direction it has to go, and naturally stop when it reaches the target. The challenging part is to decide in what direction our robot has to turn to always have a wall to the left. If our robot has a wall to the left and then takes a step forward, there are four cases to consider:

1. There is no wall to the left any more: If that happens we know that the left wall takes a turn to the left and the robot needs to follow it so it turns left.
2. There is a wall to the left and no wall to the front: The robot does not

need to turn since the way is free and there is a wall to the left.

3. There is a wall to the left, a wall to the front and no wall to the right: The robot turns right and the wall that was previously in front of it becomes the new left wall.

4. There is a wall to the left, right and front: The robot entered a dead end and needs to turn back.

This is certainly not an efficient algorithm, because of the limited space. Especially if we consider that the robot turns and walks first before checking if it is on the target, so if the target is in the first tile, the robot has to walk the whole maze until it arrives back at the beginning.

Location	Contents	Meaning
0	22	trn 2
1	77	jnw 7
2	26	trn 6
3	77	jnw 7
4	26	trn 6
5	77	jnw 7
6	24	trn 4
7	11	fwd 1
8	50	jnt 0
9	00	hlt